Harvest Now, Decrypt Later Attacks on TLS and the Migration to ML-KEM

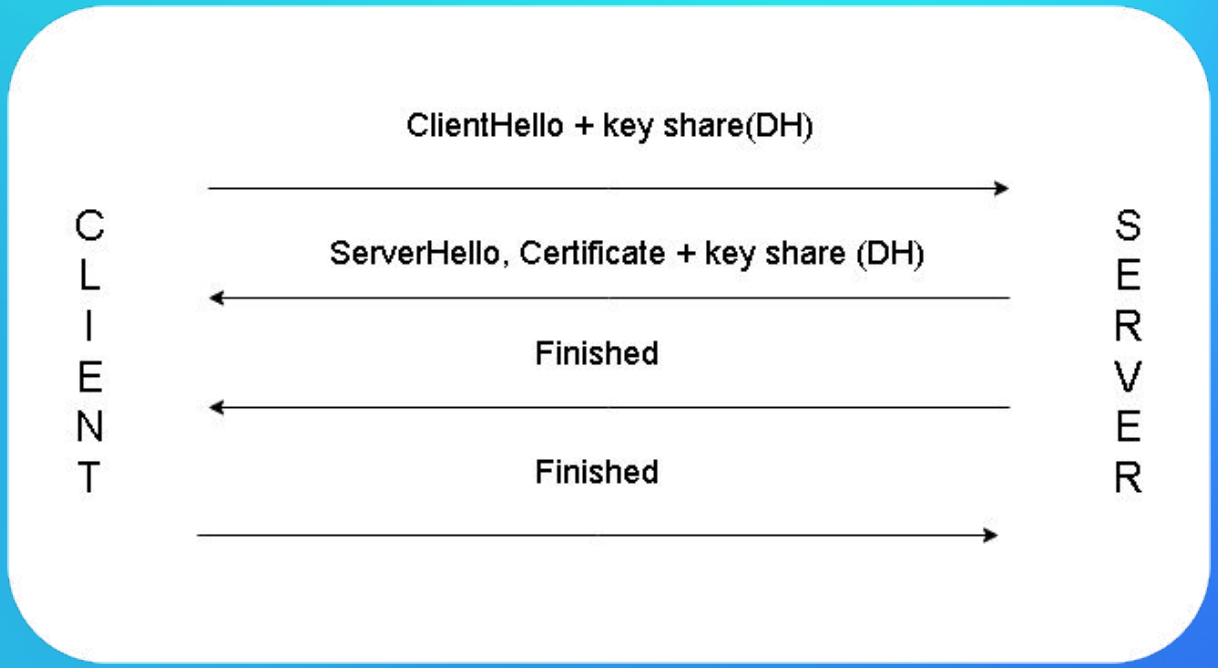
Dogăreci Bianca-Alexandra, Iscru Bianca

University of Bucharest, Faculty of Mathematics and Informatics

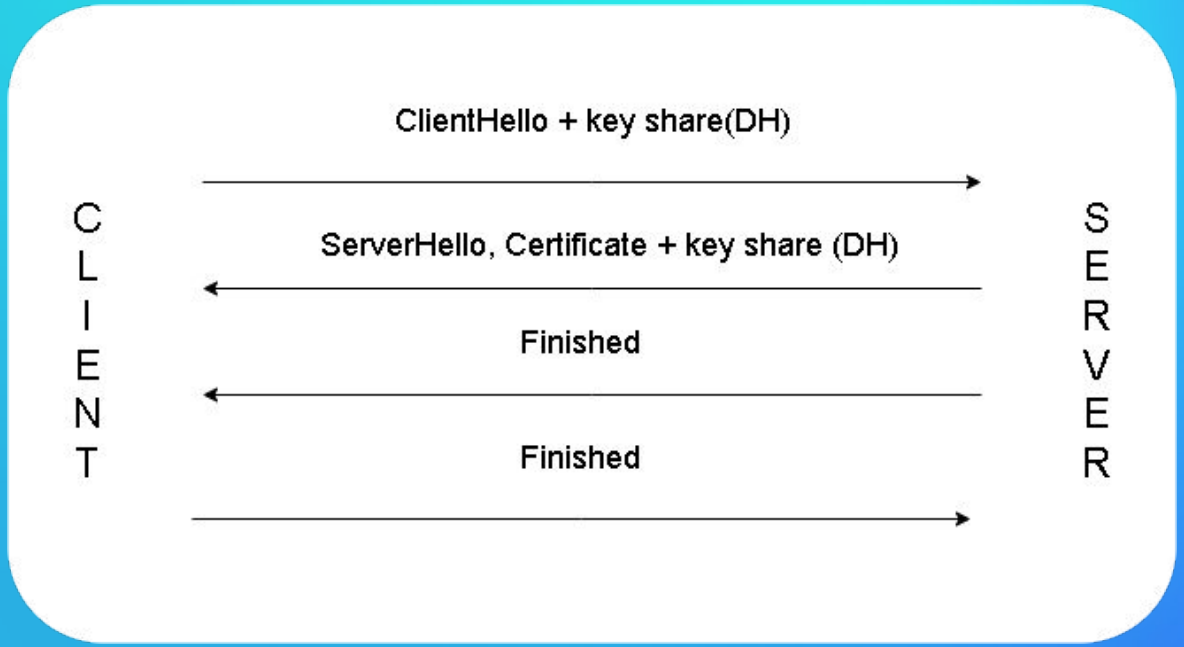
TLS 1.3 Overview

Cryptographic protocol securing the majority of Internet traffic, providing encrypted and authenticated communication between two parties over a network.

Begins with a handshake:



Handshake → Target



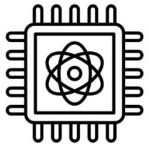
01 The Harvest-Now, Decrypt-Later Attack

Attack that proceeds in two temporally separated phases:

Now: Harvest (Record)

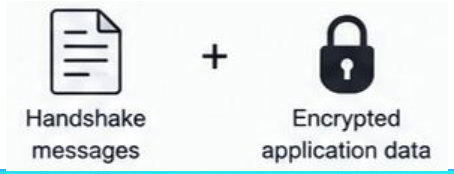
An adversary with access to the network can store TLS handshakes and encrypted traffic

Later: Decrypt



When a sufficiently powerful quantum computer becomes available, the attacker can break the handshake and decrypt the data

Store Data



02 The HN-DL Threat to TLS

? How is the TLS handshake compromised

⇨ By Shor's quantum algorithm

It solves hard number-theoretic problems, such as integer factorization and the discrete logarithm problem

Why Forward Secrecy Does Not Help

Forward secrecy:

- there's no longer a single secret value that will decrypt multiple sessions
- each connection uses freshly generated ephemeral keys

However, in an HN-DL attack, this means the adversary must break the ephemeral Diffie-Hellman key exchange for each targeted session independently, rather than relying on a single long-term key.

03

Migrating to Post-Quantum TLS with ML-KEM

ML-KEM

- Standardised by NIST as FIPS 203 (August 2024)
- Based on the Module-LWE problem → resistant to quantum computers
- Replaces ECDHE in the TLS handshake
- Already in production: Google Chrome, Cloudflare, AWS

How ML-KEM works

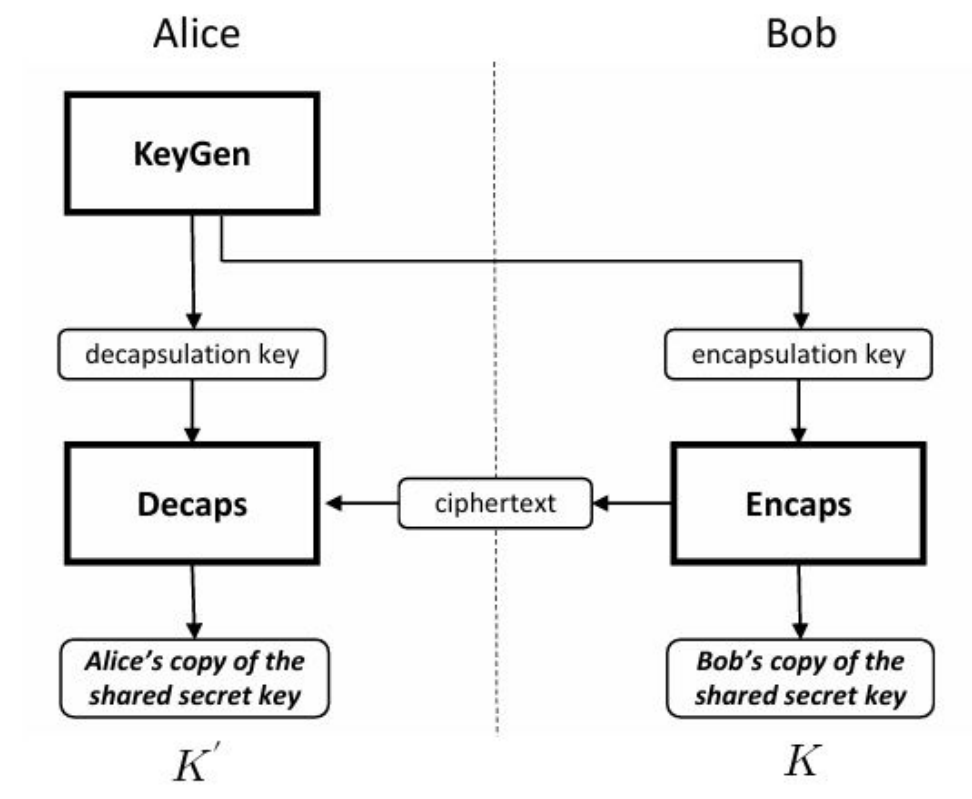


Figure 1- NIST FIPS 203

04

Experimental Evaluation of Classical vs Post-Quantum Handshake

Test Environment & System Specifications

Component	Details
CPU	Intel Core processor (Intel64 Family 6 Model 165)
RAM	16 GB
OS	Windows 11 (build 26200)*
Iterations	100 per variant
Timing	time.perf_counter() — microsecond precision

*Python reports "Windows 10" for Windows 11 because Microsoft kept the version number as 10.0 for API compatibility. Build numbers ≥ 22000 indicate Windows 11.

What We Measured

We measured only operations that differ between classical and post-quantum. Not Measured: HKDF (HMAC-based Extract-and-Expand Key Derivation Function), AES-GCM

Classical X25519 (Diffie-Hellman)

Phase	Operation
1	Key Generation (client + server generate key pairs)
2	Shared Secret Derivation (client + server)

```python
def measure_classical_handshake(iterations=100):
    # Measure X25519 performance - how long cryptographic operations take
    keygen_times = [] # store timing for key generation
    derive_times = [] # store timing for shared secret derivation

    for _ in range(iterations):
        # PART 1: KEY GENERATION
        # Each side (client & server) generates their own key pair
        # Private key = secret, Public key = sent over network
        start = time.perf_counter()

        client_priv = X25519PrivateKey.generate()
        client_pub = client_priv.public_key()
        server_priv = X25519PrivateKey.generate()
        server_pub = server_priv.public_key()

        keygen_time = (time.perf_counter() - start) * 1_000_000
        keygen_times.append(keygen_time)

        # PART 2: SHARED SECRET DERIVATION
        # After exchanging public keys, each side computes the same shared secret
        # "Diffie-Hellman" math
        start = time.perf_counter()

        # Client computes secret using his private key + server's public key
        client_shared = client_priv.exchange(server_pub)
        # Server computes secret using his private key + client's public key
        server_shared = server_priv.exchange(client_pub)
        # If correct, client_shared == server_shared
        # Session key (HKDF) will be derived from this secret later

        derive_time = (time.perf_counter() - start) * 1_000_000
        derive_times.append(derive_time)

    return {
        'keygen_us': statistics.mean(keygen_times),
        'keygen_std': statistics.stdev(keygen_times),
        'derive_us': statistics.mean(derive_times),
        'derive_std': statistics.stdev(derive_times),
    }
```

What We Measured

ML-KEM-768 (Key Encapsulation Mechanism)

Phase	Operation
1	Key Generation (server)
2	Encapsulation (client creates secret + ciphertext)
3	Decapsulation (server recovers secret from ciphertext)

```python
def measure_mlkem_handshake(iterations=100):
    # Measure ML-KEM-768 performance - how long post-quantum crypto operations take
    # ML-KEM is a Key Encapsulation Mechanism (KEM), different from Diffie-Hellman
    keygen_times = [] # store timing for key pair generation
    encaps_times = [] # store timing for encapsulation (client creates ciphertext)
    decaps_times = [] # store timing for decapsulation (server recovers secret)

    for _ in range(iterations):
        # PART 1: KEY GENERATION
        # Server generates a key pair (different from classical DH)
        # - Public key (encapsulation key): can be shared publicly
        # - Private key (decapsulation key): must stay secret
        start = time.perf_counter()

        # public key size: 1184 bytes
        # private key size: 2400 bytes
        keypair = KEM.generate_keypair("ML-KEM-768")

        keygen_time = (time.perf_counter() - start) * 1_000_000
        keygen_times.append(keygen_time)

        # PART 2: ENCAPSULATION (Client side)
        # Client takes server's public key and "encapsulates" a shared secret
        start = time.perf_counter()

        # - ciphertext: sent to server over network (1088 bytes)
        # - shared_secret: kept by client, used for AES encryption
        ciphertext, shared_secret = KEM.encapsulate("ML-KEM-768", keypair.public_key)

        encaps_time = (time.perf_counter() - start) * 1_000_000
        encaps_times.append(encaps_time)

        # PART 3: DECAPSULATION (Server side)
        # Server decapsulates using private key + ciphertext from client
        # Recovers the SAME shared secret that client created
        start = time.perf_counter()

        recovered_secret = KEM.decapsulate("ML-KEM-768", keypair.private_key, ciphertext)

        decaps_time = (time.perf_counter() - start) * 1_000_000
        decaps_times.append(decaps_time)

        # After this, both parties have the same shared_secret
        # They will both derive the AES session key using HKDF (same as classical)

    return {
        'keygen_us': statistics.mean(keygen_times),
        'keygen_std': statistics.stdev(keygen_times),
        'encaps_us': statistics.mean(encaps_times),
        'encaps_std': statistics.stdev(encaps_times),
        'decaps_us': statistics.mean(decaps_times),
        'decaps_std': statistics.stdev(decaps_times),
    }
```

Results - Handshake Size

Message	X25519	ML-KEM-768
Client → Server	32 bytes	1,088 bytes
Server → Client	32 bytes	1,184 bytes
Total	**64 bytes**	**2,272 bytes**
Increase	**1.0x**	**35.5x**

Each message < Ethernet MTU (1,500 bytes) → No fragmentation

Parallelism vs Sequential

Our measurements report **total CPU work from both parties (client and server) combined**.

Protocol	Real Deployment Behaviour	Real-Clock Latency
X25519	Parallel (keygen for client and server + secret derivation for client and server happen simultaneously)	**Half our measurement** (~72 µs)
ML-KEM-768	Sequential (server must finish keygen before client encapsulates and client must finish encapsulation before server decapsulates)	**Matches our measurement** (~448 µs)

Results - Computational Overhead

Operation	X25519	ML-KEM-768
Key generation	82 ± 14 µs	100 ± 22 µs
Secret derivation	62 ± 3 µs	348 ± 66 µs
Total handshake	144 ± 17 µs	448 ± 88 µs

Metric	Value
Slowdown	3.1x
Classical	0.144 ms
ML-KEM	0.448 ms

Our 3.1x slowdown is a LOWER BOUND. The actual difference may be ~6.2x!

Network Security Implications - Wire Size

Impact	Explanation
35.5× bandwidth increase	64 bytes → 2,272 bytes per handshake
No fragmentation	Each message fits in 1,500 byte MTU
Concern	High-traffic servers (1M handshakes/sec): 64 MB/s → 2.27 GB/s

Network Security Implications - DoS Vulnerability

Metric	X25519	ML-KEM-768
CPU time per handshake	144 µs	448 µs
Max handshakes/sec (100% CPU)	~6,900	~2,200

Attacker needs 3.1× fewer requests to exhaust server CPU!

Denial of Service (DoS) attack surface increases by factor of 3.1×!

Network Security Implications - IoT Devices

Problem: IoT devices have limited computational resources.

Device Type	Typical CPU Internal Clock Ticks	Estimated Handshake Time
Normal computer	2-4 GHz (2,000,000,000 / 4,000,000,000)	0.448 ms
Smart sensor	100 MHz (100,000,000 cycles/ticks per second)	~10 ms
Smart light bulb	50 MHz (50,000,000 cycles/ticks per second)	~20-50 ms

- ML-KEM requires more mathematical operations than classical X25519
- On a slow CPU, these operations take longer
- IoT devices use aggressive power-saving timers (short timeouts)
- If the handshake exceeds the timeout, the connection fails

Result: Existing IoT devices may require hardware upgrades or will lose network connectivity after post-quantum migration.

Bibliography

- Amazon Web Services: AWS application and network load balancers now support post-quantum key exchange for TLS. AWS What's New (Nov 2025)
- Blanco-Romero, J., Mendoza, F.A., Rubio, C.G., Campo, C., Sánchez, D.D.: On the practical feasibility of harvest-now, decrypt-later attacks (2026)
- Chen, L., Jordan, S., Liu, Y.K., Moody, D., Peralta, R., Perlner, R., Smith-Tone, D.: Report on post-quantum cryptography. Tech. rep. (Apr 2016)
- Mallick, T., Kundu, A., Kompella, R.: Study of post quantum status of widely used protocols (2026)
- National Institute of Standards and Technology: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM). Federal Information Processing Standard FIPS 203, NIST (2024)
- Shor, P.W.: Algorithms for quantum computation: Discrete logarithms and factoring. In: Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS). pp. 124–134. IEEE (1994)

Thank you for your attention!